Работа с файлами в стиле C

```
#define _CRT_SECURE_NO_WARNINGS
```

Зачем это

Многие программисты до сих пор используют функции из языка C, так как они работают быстрее, чем потоки iostream/fstream, и потребляют меньше памяти. Работа с файлами через библиотеку <cstdio> (унаследованную из языка C) завязана на использовании структуры FILE.

Доступ к файлу осуществляется не через объекты-потоки, а через указатель на эту структуру (FILE*), который называют файловым дескриптором или указателем на файл.

Открытие и закрытие файла

Для открытия используется функция fopen, а для закрытия — fclose.

```
FILE* fopen(const char* имя_файла, const char* режим);

int fclose(FILE* stream);
```

Основные режимы открытия

- "r" (Read) — открыть для чтения. Файл должен существовать, иначе функция вернет nullptr.
- "w" (Write) — открыть для записи. Если файла нет, он создастся. Если есть — его содержимое полностью удалится (очистится).
- "a" (Append) — открыть для дозаписи. Все новые данные будут добавляться в конец файла.
- "r+", "w+", "a+" — смешанные режимы, позволяющие одновременно и читать, и писать.

Пример

```
FILE* f = fopen("data.txt", "r");

if (f == nullptr) {

    // Проверка на существование файла

    cout <<  "Ошибка открытия файла";

    return 1;

}

// ... работа с файлом ...

fclose(f); // Закрытие файла обязательно
```

Функции записи данных

1. Форматированный вывод: fprintf

Аналог функции printf, но первым аргументом принимает указатель на файл. Подходит для создания структурированных отчетов или таблиц:

```
int age = 25;

double salary = 1500.50;

fprintf(f, "Имя: %s, Возраст: %d, Зарплата: %.2f\n",
"Иван", age, salary);
```

Функции записи данных

2. Запись строки: fputs

Просто записывает строку в файл:

```
fputs("Простая строка текста.\n", f);
```

3. Посимвольный вывод: fputc

Записывает в файл один конкретный символ.

```
fputc('A', f);
```

Функции чтения данных

1. Построчное чтение: fgets. Читает строку из файла до тех пор, пока не встретит символ переноса строки \n, конец файла EOF или пока не заполнится буфер.

```
char buffer[256];

// Читаем не более 255 символов в массив buffer из файла f

while (fgets(buffer, sizeof(buffer), f) != nullptr) {

    printf("Строка: %s", buffer);

}
```

Функции чтения данных

2. Форматированное чтение: fscanf

Аналог scanf. Удобно, если в файле записаны чистые данные (например, столбцы чисел или слов), разделенные пробелами.

```
char name[50];

int age;

// Читает слово в name и число в age. Прекращает цикл, если файл закончился

while (fscanf(f, "%49s %d", name, &age) == 2) {

    printf("Прочитано: %s (%d лет)\n", name, age);

}
```

Функции чтения данных

Посимвольное чтение: fgetc

Возвращает один символ из файла. При достижении конца файла возвращает специальную константу EOF (End of File).

```
int ch; // Используем int, а не char, чтобы корректно поймать EOF

while ((ch = fgetc(f)) != EOF) {

    putchar(ch); // Вывод символа на экран

}
```

Считывание всего файла в память

Если файл небольшой, проще и быстрее не читать его построчно, а загрузить целиком в одну строку string или буфер строк.

Считывание всего файла в память

```
#include <iostream>
#include <fstream>
#include <sstream>
using namespace std;
int main() {
    ifstream in("example.txt");
    if (in.is_open()) {
        stringstream buffer;
        buffer << in.rdbuf(); // Перенаправляем весь буфер файла в строку

        string full_text = buffer.str();
        cout << full_text;
    }
    return 0;
}
```

Использование std::filesystem (C++17)

С самого C++17 появился мощный инструмент для управления файловой системой. Он используется не для чтения текста внутри файла, а для управления самим файлом.

Использование std::filesystem (C++17)

```
#include <iostream>
#include <filesystem>

namespace fs = std::filesystem;

int main() {
    fs::path p("example.txt");

    if (fs::exists(p)) {
        std::cout << "Размер файла: " << fs::file_size(p) << " байт\n";
    } else {
        std::cout << "Файл не найден\n";
    }

    // Можно легко копировать, переименовывать или удалять:
    // fs::copy("example.txt", "copy.txt");
    // fs::remove("copy.txt");
}
```

Работа с бинарными файлами

Работа с бинарными файлами производится посредством посимвольного чтения-записи. Поскольку один объект `char` соответствует одному байту, то фактически речь идет о побайтовых записи/чтения

Запись файла

```c
#include <stdio.h>

int main(void)
{
    // записываемая строка
    char message[] = "Hello!";
    // файл для записи
    char filename[] = "data.bin";
    // находим длину записываемой строки
    int n = sizeof(message)/sizeof(message[0]);

    FILE * fp= fopen(filename, "w");
    if(fp)
    {
        // посимвольно записываем в файл
        for(int i=0; i < n; i++)
        {
            putc(message[i], fp);
        }

        fclose(fp);
        printf("File has been written\n");
    }
}
```

Чтение файла

```c
#include <stdio.h>

int main(void)
{
    // символ для считывания
    char c;
    // файл для чтения
    char filename[] = "data.txt";

    FILE * fp= fopen(filename, "r");
    if(fp)
    {
        // считываем посимвольно из файла
        while((c=getc(fp))!= EOF)
        {
            printf("%c", c);
        }
        fclose(fp);
    }
}
```